

Linear Programming Course Summary

Generated by Scribe.AI

December 3, 2024

1 Key Terms

- **Slack Variable:** A variable added to a less-than-or-equal-to inequality constraint to transform it into an equality constraint. The slack variable represents the difference between the left and right sides of the inequality.
- **Feasible Region:** The set of all points that satisfy all the constraints of a linear programming problem.
- **Vertex:** A point where two or more constraint boundaries intersect in the feasible region of a linear programming problem. In the Simplex method, optimal solutions are found at vertices.
- **Basic Variable:** In the Simplex method, a variable that is expressed in terms of non-basic variables in a dictionary.
- **Non-basic Variable:** In the Simplex method, a variable set to zero in a given iteration of the algorithm.
- **Dictionary:** A representation of a linear programming problem where basic variables are expressed as functions of non-basic variables. Used in the Simplex method to iteratively find the optimal solution.
- **Auxiliary Problem:** A modified linear programming problem introduced to find a feasible starting point for the Simplex method when the origin is not feasible in the original problem.
- **Largest-Coefficient Rule:** A pivot rule in the simplex method that selects the entering variable with the largest coefficient in the objective function row of the dictionary.
- **Largest-Increase Rule:** A pivot rule in the simplex method that selects the entering variable that produces the largest increase in the objective function value.
- **Klee-Minty Example:** A worst-case example for the simplex method, demonstrating exponential time complexity for certain pivot rules.
- **Bland's Rule:** A pivot rule in the simplex method designed to avoid cycling, ensuring termination.
- **Lexicographic Method:** A method for choosing the leaving variable in the simplex method to prevent cycling.
- **Smoothed Complexity:** A measure of an algorithm's complexity that considers the effect of small random perturbations to the input.
- **Interior Method:** A class of linear programming algorithms that traverse the interior of the feasible region, in contrast to the simplex method which follows the boundary.
- **Primal Problem:** A linear programming problem formulated to maximize an objective function subject to constraints and non-negativity conditions.
- **Dual Problem:** A linear programming problem derived from the primal problem, formulated to minimize an objective function subject to constraints and non-negativity conditions. Solving the dual problem provides insights into the solution of the primal problem.

- **Weak Duality:** A theorem stating that the objective function value of the primal problem is always less than or equal to the objective function value of the dual problem for any feasible solutions to both problems.
- **Negative Transpose Property:** The property that the coefficient matrix of the dual problem is the negative transpose of the coefficient matrix of the primal problem. This property is preserved throughout the simplex method iterations.
- **Complementary Variables:** Pairs of primal and dual variables (and their corresponding slack variables) that exhibit a complementary relationship in the optimal solution; at optimality, at least one variable in each pair must be zero.
- **Strong Duality Theorem:** A theorem stating that if a linear programming problem has an optimal solution, then its dual problem also has an optimal solution, and the optimal objective function values of both problems are equal.
- **Complementary Slackness:** A condition stating that for optimal primal and dual solutions, the product of each primal variable and its corresponding dual slack variable is zero, and the product of each dual variable and its corresponding primal slack variable is zero.
- **Lagrange Multiplier:** A variable introduced in the Lagrangian function to incorporate constraints into an optimization problem.
- **Lagrangian Function:** A function that combines the objective function and constraints of an optimization problem using Lagrange multipliers.
- **Weak Duality Theorem:** A theorem stating that the optimal value of the primal problem is always less than or equal to the optimal value of the dual problem.
- **Diet Problem:** A classic linear programming problem where the goal is to minimize the cost of a diet while meeting minimum nutritional requirements. Slides 2, 3, 4, 5, 6, 7, 8
- **Matrix Notation for Linear Programming:** A representation of a linear programming problem using matrices and vectors, where the objective function and constraints are expressed in a compact form.
- **Simplex Iterates:** The sequence of solutions generated by the simplex method during its iterative process of moving from one feasible solution to another, improving the objective function value at each step.
- **Augmented Matrix:** A matrix formed by adding a column of constants (the right-hand side of the constraints) to the coefficient matrix of a system of linear equations.
- **Basic Variables:** In the simplex method, a subset of variables whose values are determined by solving a system of linear equations formed by setting the non-basic variables to zero.
- **Non-Basic Variables:** In the simplex method, the variables that are set to zero to solve for the basic variables.
- **Partitioning of Variables:** The process of separating the variables in a linear programming problem into basic and non-basic variables.
- **Simplex Tableau:** A tabular representation of the simplex method's iterations, showing the current values of the basic and non-basic variables, the objective function value, and the coefficients of the variables in the constraints.
- **Entering Variable:** The non-basic variable selected to become a basic variable in the next iteration of the simplex method.
- **Leaving Variable:** The basic variable selected to become a non-basic variable in the next iteration of the simplex method.

- **Optimal Solution:** A feasible solution to a linear programming problem that maximizes (or minimizes) the objective function.
- **Reduced Costs:** The values that indicate how much the objective function value would change if a non-basic variable were to become basic. A positive reduced cost indicates that increasing the corresponding non-basic variable would decrease the objective function value.
- **Parametric Analysis:** A technique used to study how changes in the parameters of a linear programming problem (objective function coefficients or constraint values) affect the optimal solution.
- **Sensitivity Analysis:** The process of determining the range of changes in the parameters of a linear programming problem for which the current optimal solution remains optimal.
- **Shadow Price:** The rate of change in the optimal objective function value for a unit change in the right-hand side of a constraint.
- **Basis Matrix:** A square matrix formed by the columns of the constraint matrix corresponding to the basic variables in a linear programming problem.
- **Non-basis Matrix:** A matrix formed by the columns of the constraint matrix corresponding to the non-basic variables in a linear programming problem.
- **Convex Set:** A set $S \subseteq R^m$ such that for any $z_1, z_2 \in S$, $\lambda z_1 + (1 - \lambda)z_2 \in S$ for $0 \leq \lambda \leq 1$.
- **Convex Combination:** A weighted sum of points $z_1, z_2, \dots, z_n \in R^m$ given by $\sum_{i=1}^n \lambda_i z_i$, where $\lambda_i \geq 0$ and $\sum_{i=1}^n \lambda_i = 1$.
- **Convex Hull:** The smallest convex set containing a given set $S \subseteq R^m$, denoted as $Conv(S) = \bigcap \{C \text{ convex} | S \subseteq C\}$.
- **Caratheodory Theorem:** If $z \in Conv(S)$, then z can be written as a convex combination of at most $m + 1$ points from S .
- **Separation Theorem:** If P and $\bar{P}$ are two disjoint nonempty polyhedra in R^n , then there exist disjoint half-spaces H and $\bar{H}$ such that $P \subset H$ and $\bar{P} \subset \bar{H}$.
- **Farkas' Lemma:** The system $Ax \leq b$ has no solution if and only if there exists a vector $y \geq 0$ such that $y^T A = 0$ and $y^T b < 0$.
- **L² Regression (Least Square):** A regression method that minimizes the sum of squared differences between observed and predicted values.
- **L¹ Regression:** A regression method that minimizes the sum of absolute differences between observed and predicted values.
- **L Regression:** A regression method that minimizes the maximum absolute difference between observed and predicted values.
- **Linear Separability:** The property of a dataset where a hyperplane can perfectly separate data points into different classes.
- **Maximum Margin Classifier:** A classifier that aims to maximize the minimum distance between the separating hyperplane and the closest data points.
- **Support Vector Machine (SVM):** A classification algorithm that finds the optimal hyperplane that maximizes the margin between classes.
- **Support Vectors:** The data points closest to the hyperplane in an SVM; they are crucial in determining the hyperplane.
- **Lagrangian Formulation of SVM:** A method to reformulate the SVM optimization problem using Lagrange multipliers, often leading to a dual problem.

- **Geometric Interpretation of SVM:** Understanding SVM by visualizing the hyperplane and its margin in the data space.
- **SVM Optimization as a QP Problem:** Formulating the SVM problem as a quadratic programming problem to find the optimal hyperplane.
- **Maximum Inscribed Circle in a Convex Polyhedron:** Finding the largest circle that can fit entirely within a convex polyhedron.
- **Distance of a Point from a Hyperplane:** The formula to calculate the perpendicular distance between a point and a hyperplane.
- **Smallest Ball Problem:** Finding the smallest-radius sphere that encloses a given set of points in a d -dimensional space.
- **Production Planning:** The process of determining optimal production levels over a given time period to meet predicted demand while minimizing costs.
- **Average Demand:** The mean of the predicted demand over a given time period.
- **Cost of Changing Production:** The cost associated with altering production levels from one period to the next.
- **Cost of Storage:** The cost associated with storing surplus inventory.
- **Workforce Planning:** The process of determining the optimal number of employees of different types to meet fluctuating labor demands while minimizing costs.
- **Portfolio Selection (Markowitz Model):** A financial model that aims to optimize investment portfolios by maximizing expected return while minimizing risk.
- **Maximum Weight Matching:** A combinatorial optimization problem that involves finding a matching in a weighted graph that maximizes the total weight of the edges in the matching.
- **Machine Scheduling:** The problem of assigning jobs to machines to minimize the total completion time.
- **Knapsack Problem:** A combinatorial optimization problem where the goal is to select a subset of items with maximum total value, subject to a weight constraint.
- **Cutting Stock Problem:** An optimization problem that involves determining how to cut larger rolls of material into smaller rolls of specified widths to minimize waste.
- **Sparse Solutions of Linear System:** Finding a solution to an underdetermined system of linear equations that has a limited number of non-zero elements.
- **Farkas Lemma:** A fundamental theorem in linear programming that provides a condition for the infeasibility of a system of linear inequalities. It states that a system of inequalities $Ax \leq b$ has no solution if and only if there exists a vector y such that $A^T y = 0$, $y \geq 0$, and $b^T y < 0$.
- **Fredholm Alternatives:** A statement of the two mutually exclusive possibilities regarding the solvability of a linear system $Ax = b$. Either the system has a solution, or there exists a vector y such that $y^T A = 0$ and $y^T b \neq 0$.
- **Integer Programming:** A mathematical optimization problem where the objective function and constraints are linear, but the variables are restricted to integer values.
- **Relaxed Problem:** The linear programming problem obtained by removing the integer constraints from an integer programming problem.
- **Branch-and-Bound:** A method for solving integer programming problems by recursively partitioning the feasible region and bounding the objective function value.

- **Depth-First Search:** A search strategy for exploring the enumeration tree in Branch and Bound, prioritizing deeper branches before wider ones.
- **Pruning:** The process of eliminating branches in the enumeration tree of a Branch and Bound algorithm that cannot lead to a better solution than one already found.
- **Gomory Cuts:** Additional linear constraints added to an integer programming problem to cut off non-integer solutions from the feasible region of the relaxed problem.
- **Network:** A graph (or digraph) consisting of a set of nodes (N) and a set of directed arcs (A).
- **Node:** A point in a network representing a location or entity.
- **Arc:** A directed edge connecting two nodes in a network, often associated with a capacity or cost.
- b_i : The net flow at node i ; positive values indicate supply, negative values indicate demand.
- x_{ij} : The amount of flow transported from node i to node j .
- c_{ij} : The cost of transporting one unit of flow from node i to node j .
- **Balanced Equation:** An equation stating that the sum of flows entering a node plus the supply at that node equals the sum of flows leaving the node.
- **Incidence Matrix (A):** A matrix defining the flow relationships between nodes and arcs in a network.
- **Cost Vector (c):** A vector containing the cost of transporting one unit of flow along each arc.
- **Supply/Demand Vector (b):** A vector representing the net flow at each node; positive values indicate supply, negative values indicate demand.
- $A_{ij}(k,l)$: An element of the incidence matrix, defined based on the relationship between the row index (i) and column indices (k and l).
- **Rank(A):** The rank of the incidence matrix, indicating the number of linearly independent equations in the system.
- **Primal Problem (P):** The original minimization problem in a linear program.
- **Dual Problem (D):** The maximization problem associated with a primal problem in a linear program.
- **Path:** A sequence of nodes where each consecutive pair is connected by an arc in the network.
- **Cycle:** A path that starts and ends at the same node.
- **Tree:** A connected graph with no cycles.
- **Spanning Tree:** A tree that includes all nodes of the network.
- **Tree Solution:** A solution where the flow along arcs in the spanning tree is zero.
- **Root Node:** A node selected to simplify the representation of the flow equations.
- **Basis Matrix (B):** A square submatrix of the constraint matrix that forms a basis for the solution space.
- **Non-basis Matrix (N):** The remaining columns of the constraint matrix after selecting the basis matrix.
- **Reduced Cost:** The cost of increasing a non-basic variable by one unit.
- **Dual Slack:** The difference between the cost of an arc and the difference in dual variables at its endpoints.
- z_{ij} : The dual slack associated with arc (i,j) .
- y_i : The dual variable associated with node i .

- $\tilde{X}$: The vector of primal flow variables.
- μ : Perturbation parameter in the parametric self-dual method.
- **Transportation Problem**: A network flow problem where the total flow out of each supply node is less than or equal to the supply, and the total flow into each demand node is greater than or equal to the demand.
- **Dummy Node**: A node added to a network flow problem to handle inequality constraints, representing excess supply or unmet demand.
- **Upper Bounded Transshipment Problem**: A network flow problem where the flow on each arc is bounded above by a specified upper bound.
- **Shortest Path Problem**: Finding the path with the minimum total cost or distance between a source node and a destination node in a network.
- **Bellman's Equation**: A recursive equation used to solve the shortest path problem by iteratively calculating the shortest distance from each node to the destination node.
- **Dijkstra's Algorithm**: An algorithm for finding the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights.
- **Max Flow**: The maximum amount of flow that can be sent from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Cut**: A partition of the nodes in a network into two disjoint sets, one containing the source node and the other containing the sink node.
- **Cut Capacity**: The sum of the capacities of all edges that cross a cut from the source set to the sink set.
- **Max-Flow Min-Cut Theorem**: The theorem stating that the maximum flow in a network is equal to the minimum capacity of any cut in the network.
- **Augmenting Path**: A path from the source to the sink in a network along which the flow can be increased.
- **Ford-Fulkerson Algorithm**: An algorithm for finding the maximum flow in a network by iteratively finding and augmenting along augmenting paths.
- **Fictitious Arc**: An artificial edge added to a network flow problem to facilitate the linear programming formulation.
- b_j : A value associated with flow constraints at node j in a network flow problem.

2 Problem Types

- **Finding an Initial Feasible Solution**: The problem of finding a feasible solution to start the Simplex method is addressed, particularly when the origin is not feasible. An auxiliary problem is introduced to find a feasible starting point.
- **Handling Infeasible Origins**: This problem focuses on situations where the origin $(0,0)$ is not within the feasible region of the linear programming problem, requiring a different approach to initiate the Simplex method.
- **Optimizing with Three Variables**: This problem demonstrates the application of the Simplex method to a linear programming problem with three variables, requiring a more complex iterative process.

- **Using Dictionaries to Represent Solutions:** This problem involves representing the linear programming problem and its solutions using dictionaries, which are systems of equations expressing basic variables in terms of non-basic variables. The process of pivoting between dictionaries is highlighted.
- **Analyzing the Efficiency of the Simplex Method:** This problem involves analyzing the number of iterations required by the Simplex method to solve linear programming problems, considering both average-case and worst-case scenarios. The analysis includes comparing different pivot rules (largest-coefficient, largest-increase) and their impact on the number of iterations.
- **Understanding Worst-Case Behavior of the Simplex Method:** This problem focuses on understanding the worst-case performance of the Simplex method, using the Klee-Minty example to illustrate how the number of iterations can grow exponentially with the problem size. The analysis includes exploring the impact of different pivot rules and variable scaling on the number of iterations.
- **Evaluating Alternative Pivot Rules:** This problem involves comparing different pivot rules for the Simplex method, such as the largest-coefficient rule, the largest-increase rule, and Bland's rule, to determine their efficiency in terms of the number of iterations and total computation time. The analysis considers the impact of scaling on the performance of these rules.
- **Empirical Analysis of Simplex Method Performance:** This problem involves analyzing empirical data from simulations to understand the average-case performance of the Simplex method. The analysis includes visualizing the relationship between problem size and the number of iterations required, and identifying trends in the data.
- **Theoretical Bounds on Simplex Method Performance:** This problem involves exploring theoretical bounds on the number of iterations required by the Simplex method, considering both deterministic and randomized pivot rules. The analysis includes discussing the limitations of current theoretical results and the concept of smoothed complexity.
- **The Existence of Polynomial-Time Algorithms for Linear Programming:** This problem involves understanding the existence of polynomial-time algorithms for linear programming, such as interior-point methods, and contrasting their theoretical performance with the Simplex method.
- **Formulating Primal and Dual Problems:** This problem involves expressing a given linear programming problem in both primal and dual forms, highlighting the relationship between the objective functions and constraints of the two formulations.
- **Applying Weak Duality Theorem:** This problem focuses on utilizing the weak duality theorem to establish an inequality relationship between the objective function values of feasible solutions to the primal and dual problems.
- **Demonstrating the Negative Transpose Property:** This problem involves showing how the negative transpose of the primal problem's coefficient matrix yields the dual problem's coefficient matrix, and how this property is preserved throughout the simplex method iterations.
- **Proving the Strong Duality Theorem:** This problem centers on proving that if the primal problem has an optimal solution, then the dual problem also has an optimal solution, and their objective function values are equal.
- **Applying the Complementary Slackness Theorem:** This problem focuses on using the complementary slackness theorem to determine the optimality of feasible solutions to the primal and dual problems by checking the conditions on the products of primal variables and dual slack variables, and vice versa.
- **Formulating the Lagrangian for problems with equality constraints:** This problem involves constructing the Lagrangian function to incorporate equality constraints into the optimization problem using Lagrange multipliers.

- **Formulating the Lagrangian for problems with inequality and equality constraints:** This problem extends the previous one by including inequality constraints in addition to equality constraints, requiring the use of Lagrange multipliers for both types of constraints.
- **Establishing weak duality using the Lagrangian:** This problem focuses on proving the weak duality theorem, which states that the optimal value of the primal problem is always less than or equal to the optimal value of the dual problem, using the Lagrangian function.
- **Deriving the dual problem from the Lagrangian:** This problem involves deriving the dual problem from the Lagrangian function by analyzing the maximum value of the Lagrangian with respect to the primal variables.
- **Establishing strong duality:** This problem focuses on proving the strong duality theorem, which states that under certain conditions, the optimal values of the primal and dual problems are equal.
- **Applying complementary slackness:** This problem involves applying the complementary slackness theorem, which states that for optimal primal and dual solutions, the product of corresponding primal and dual variables is zero.
- **Formulating the Primal and Dual Problems for the Diet Problem:** This problem involves formulating both the primal (minimizing cost) and dual (maximizing profit) linear programming problems for a diet problem, given the nutritional content of foods, their costs, and minimum nutrient requirements.
- **Interpreting the Dual Problem in the Context of the Diet Problem:** This problem focuses on understanding the economic interpretation of the dual problem in the context of the diet problem, specifically relating it to the pricing of nutrient pills and profit maximization.
- **Representing the Diet Problem using Tables:** This problem involves representing the constraints and objective function of the diet problem using tables, showing the relationship between nutrients, foods, nutrient content, and costs.
- **Representing Linear Programs in Matrix Notation:** This problem focuses on expressing linear programming problems using matrices and vectors, facilitating the application of the simplex method.
- **Partitioning Variables into Basic and Non-Basic Sets:** This problem involves separating variables into basic and non-basic sets, a crucial step in the simplex algorithm for iterative solution updates.
- **Deriving the Objective Function in Matrix Form for Simplex Iterations:** This problem details the process of expressing the objective function in terms of basic and non-basic variables during simplex iterations, using matrix operations to update the objective function value efficiently.
- **Updating the Simplex Tableau Using Matrix Operations:** This problem describes the process of updating the simplex tableau using matrix operations, specifically focusing on the transformation of the objective function and constraint equations during each iteration.
- **Illustrating Complementary Slackness with Primal and Dual Variables:** This problem demonstrates the relationship between primal and dual variables, highlighting their complementary behavior during simplex iterations.
- **Solving a Linear Programming Problem Using the Simplex Method in Matrix Form:** This problem involves applying the simplex method to a specific linear programming problem, using matrix notation to track the iterative updates of the basic and non-basic variables and the objective function.
- **Solving a Linear Programming Problem Graphically:** This problem focuses on solving a linear programming problem using the graphical method, identifying the feasible region and the optimal solution point.
- **Determining the range of objective function coefficient changes that maintain optimality:** This problem investigates how much the coefficients in the objective function can change before the optimal solution changes. The analysis uses the reduced costs of the non-basic variables to determine the allowable range.

- **Determining the range of constraint value changes that maintain optimality:** This problem investigates how much the right-hand side values of the constraints can change before the optimal solution changes. The analysis uses the optimal values of the basic variables and the inverse of the basis matrix to determine the allowable range.
- **Deriving the Dual of a General Linear Program:** This problem focuses on deriving the dual linear program from a given primal linear program, particularly when the primal problem does not have non-negativity constraints on all variables. The derivation utilizes the Lagrangian function and weak duality.
- **Defining Convex Sets:** A convex set is defined as a set where any line segment connecting two points within the set is entirely contained within the set.
- **Defining Convex Combinations:** A convex combination of points is a weighted sum of those points, where the weights are non-negative and sum to one.
- **Characterizing Convex Sets via Convex Combinations:** A set is convex if and only if it contains all convex combinations of its points.
- **Defining the Convex Hull:** The convex hull of a set is defined as the smallest convex set containing the original set.
- **Characterizing the Convex Hull:** The convex hull of a set is equivalent to the set of all convex combinations of finitely many points from the original set.
- **Caratheodory's Theorem:** Any point in the convex hull of a set in R^m can be expressed as a convex combination of at most $m + 1$ points from the original set.
- **L² Regression:** Minimizing the sum of squared errors between observed and predicted values in a linear regression model.
- **Binary Classification:** Finding a hyperplane that separates labeled data points into two classes.
- **Portfolio Selection:** Optimizing investment portfolios to maximize expected return while minimizing risk, considering historical data and covariances between assets.
- **Determining the existence of a non-negative solution to ' $Ax = b$ ':** This problem investigates whether there exists a vector ' x ' with non-negative components that satisfies the equation ' $Ax = b$ '. Several equivalent conditions are presented using a vector ' y '.
- **Determining the existence of a solution to ' $Ax \leq b$ ':** This problem explores whether a solution ' x ' exists that satisfies the inequality ' $Ax \leq b$ '. Different conditions are presented based on whether a non-negative solution is required.
- **Proving Farkas' Lemma:** This problem focuses on proving the different variants of Farkas' Lemma, which provides conditions for the existence or non-existence of solutions to systems of linear inequalities and equations. The proof uses geometric interpretations and properties of convex sets.
- **Formulating Integer Programming Problems:** This problem involves defining an objective function and constraints, with the added requirement that decision variables must take on integer values.
- **Solving Integer Programming Problems by Rounding:** This problem explores the limitations of solving the relaxed linear program and then rounding the solution to obtain an integer solution. It highlights that this approach may yield infeasible solutions or solutions far from the true optimum.
- **Applying the Branch and Bound Method:** This problem focuses on using the branch and bound algorithm to solve integer programming problems by systematically exploring the solution space through branching and bounding.
- **Using Depth-First Search in Branch and Bound:** This problem discusses the advantages of using depth-first search in the branch and bound algorithm, including the likelihood of finding integer solutions deeper in the tree and the ability to prune branches.

- **Applying Gomory Cuts:** This problem involves using Gomory cuts to iteratively refine the feasible region of an integer programming problem by adding constraints derived from the fractional parts of the variables in the relaxed problem's optimal dictionary.
- **Formulating Network Flow Problems as Linear Programs:** This problem involves representing a network flow problem, with its nodes, arcs, supplies, demands, and costs, as a linear program with an objective function and constraints.
- **Analyzing the Incidence Matrix of a Network:** This problem focuses on understanding the properties of the incidence matrix, including its rank and the relationship between its submatrices and spanning trees of the network.
- **Finding a Feasible Tree Solution:** This problem involves finding a feasible solution to the network flow problem that corresponds to a spanning tree of the network, where flow is zero along arcs in the tree.
- **Applying Complementary Slackness to Network Flows:** This problem involves using the complementary slackness theorem to verify the optimality of a network flow solution, checking the relationship between primal flows and dual slack variables.
- **Using the Primal Simplex Method for Network Flows:** This problem involves applying the primal simplex method to iteratively improve a network flow solution, updating the spanning tree, primal flows, and dual variables until an optimal solution is reached.
- **Finding a Spanning Tree:** This problem involves finding a connected, acyclic subgraph that spans all nodes in a given network.
- **Finding a Primal Feasible Flow:** This problem focuses on determining a flow assignment in a network that satisfies all supply/demand constraints, even if some flow values are negative.
- **Finding a Dual Feasible Solution:** This problem involves finding a set of dual variables that satisfy the dual constraints of the network flow problem.
- **Modifying the Spanning Tree and Variables:** This problem describes the iterative process of updating the spanning tree and associated primal and dual variables to improve the solution.
- **Updating Primal and Dual Flows:** This problem involves adjusting the flow values on the edges of the network and the corresponding dual variables to maintain feasibility and improve the objective function value.
- **Selecting Entering and Leaving Arcs:** This problem focuses on choosing appropriate entering and leaving arcs during the iterative process of the network simplex method to maintain a valid spanning tree and improve the solution.
- **Proving Cost Reduction:** This problem involves demonstrating that the algorithm's selection rules guarantee a reduction in the objective function value at each iteration.
- **Handling Two Disconnected Trees:** This problem addresses the situation where the spanning tree becomes disconnected during the algorithm's execution, requiring a specific strategy to reconnect the trees.
- **Updating Dual Variables:** This problem involves updating the dual variables to maintain consistency with the updated primal solution after a pivot step.
- **Choosing an Entering Arc in the Dual Simplex Method:** This problem focuses on selecting an entering arc in the dual network simplex method that maintains dual feasibility.
- **Formulating Transportation Problems with Inequality Constraints:** This problem involves formulating a linear program to model a transportation problem where supply at sources may exceed demand at destinations, or vice versa. Dummy nodes and slack variables are introduced to handle the inequalities.

- **Formulating the Upper Bounded Transshipment Problem:** This problem involves formulating a linear program to model a network flow problem where the flow along each arc is subject to an upper bound. Slack variables are used to transform the inequality constraints into equalities.
- **Formulating the Shortest Path Problem as a Network Flow Problem:** This problem involves formulating a linear program to model the shortest path problem in a network, where the objective is to minimize the total cost of the path. The constraints ensure that exactly one unit of flow leaves the source, one unit enters the destination, and flow is conserved at intermediate nodes.
- **Solving the Shortest Path Problem using Bellman's Equation:** This problem involves using Bellman's equation to iteratively approximate the shortest distances from each node to a destination node. The algorithm involves updating distance estimates until convergence.
- **Solving the Shortest Path Problem using Dijkstra's Algorithm:** This problem involves using Dijkstra's algorithm to find the shortest paths from a source node to all other nodes in a graph. The algorithm maintains a set of processed nodes and iteratively updates shortest distance estimates.
- **Finding the Maximum Flow in a Network:** This problem involves determining the maximum amount of flow that can be sent from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Proving the Max-Flow Min-Cut Theorem:** This problem focuses on proving the theorem that states the maximum flow in a network is equal to the minimum capacity of any cut separating the source and sink nodes.
- **Implementing the Ford-Fulkerson Algorithm:** This problem involves understanding and implementing the Ford-Fulkerson algorithm, a method for finding the maximum flow in a network by iteratively finding augmenting paths.
- **Formulating the Maximum Flow Problem as a Linear Program:** This problem involves expressing the maximum flow problem as a linear programming problem, defining the objective function and constraints.
- **Finding a set of "fair prices" in a network flow problem:** This problem involves determining a set of prices y_i at each node i such that $y_i + c_{ij} = y_j$ for each arc ij in a spanning tree, representing the cost of shipping along the arc.
- **Identifying profitable shipping opportunities for a competitor:** This problem focuses on finding an arc ij in a network such that $y_i + c_{ij} < y_j$, indicating a potential for a competitor to undercut the current market prices and improve the solution.
- **Adjusting shipments to maintain feasibility after identifying a profitable opportunity:** This problem involves modifying the flow along arcs in a spanning tree to maintain feasibility after a new arc is introduced, ensuring that the supply and demand constraints are still satisfied.
- **Finding a feasible solution on a spanning tree:** This problem involves finding an initial feasible solution for a network flow problem by selecting a spanning tree that connects all nodes without forming cycles.
- **Formulating the Primal and Dual Problems:** This problem involves constructing the dual linear program from a given primal linear program, demonstrating the relationship between their objective functions and constraints.
- **Solving Linear Programs Graphically:** This problem involves using the graphical method to solve a linear program with two variables by identifying the feasible region and the optimal solution point.
- **Performing Sensitivity Analysis:** This problem explores how changes in the objective function coefficients or constraint values affect the optimal solution and the range of parameter changes that maintain optimality.

- **Interpreting Dual Variables:** This problem involves interpreting the meaning of dual variables (shadow prices) in the context of the problem, such as the marginal value of a resource.
- **Solving a Linear Program in Matrix Form:** This problem involves setting up and solving a linear program using matrix notation, including identifying the basis and non-basis matrices.
- **Geometric Interpretation of Duality:** This problem involves visualizing the relationship between the primal and dual problems geometrically in a two-dimensional space.
- **Analyzing Complementary Slackness Conditions:** This problem involves analyzing the complementary slackness conditions to determine the optimal solution and the range of parameter changes that maintain optimality.

3 Algorithm Solutions

- **Simplex Method:** Iteratively move from one vertex of the feasible region to another by setting non-basic variables to zero and choosing a new set of (non)basic variables to improve the objective function. This corresponds to moving from vertex to vertex in the feasible region.
- **Graphical Method:** Identify the feasible region defined by the constraints. The optimal solution will be at one of the vertices of the feasible region.
- **Auxiliary Problem Method:** If the origin is not feasible, introduce a new variable x_0 and create an auxiliary problem to find a feasible solution. Solve the auxiliary problem using the Simplex method. If the optimal solution has $x_0 = 0$, then this solution is feasible for the original problem.
- **Dual Problem Formulation (General LP):** Minimize $\lambda^T b$ subject to $A^T \lambda \geq c$ and $\lambda \geq 0$.
- **Simplex Method in Matrix Form:** The simplex method is implemented using matrix operations on an augmented matrix representing the constraints and objective function. The algorithm iteratively updates the basis of variables to improve the objective function value until optimality is reached.
- **Dual of General LP Problem:** Given a primal problem $\max \vec{s} = \vec{c}\vec{x}$ subject to $\vec{A}\vec{x} \leq \vec{b}$ and $\vec{x} \geq \vec{0}$, its dual is $\min \vec{s} = \vec{b}^T \vec{y}$ subject to $\vec{A}^T \vec{y} \geq \vec{c}$ and $\vec{y} \geq \vec{0}$.
- **Dual of General LP Problem (with unbounded variables):** Given a primal problem $\max \vec{c}^T \vec{x}$ subject to $\vec{A}\vec{x} \leq \vec{b}$ and $\vec{x} \leq \vec{u}$ (where $\vec{x}$ is not necessarily $\geq \vec{0}$), its dual is $\min \vec{b}^T \vec{y} + \vec{u}^T \vec{z}$ subject to $\vec{A}^T \vec{y} + \vec{z} \geq \vec{c}$ and $\vec{y}, \vec{z} \geq \vec{0}$.
- **Branch and Bound:** This algorithm solves integer programming problems by recursively partitioning the feasible region into subproblems, solving the relaxed linear program for each subproblem, and pruning branches that cannot yield a better solution than the best integer solution found so far. Depth-first search is often used to explore the enumeration tree.
- **Network Flow Problem Formulation:** Minimize $\sum_{(i,j) \in \mathcal{A}} C_{ij} X_{ij}$ subject to $\sum_i x_{ik} - \sum_j x_{kj} = -b_k$ for all nodes k and $X_{ij} \geq 0$.
- **Primal Simplex Method for Network Flows:** Iteratively identify the entering variable (arc with the most negative reduced cost), leaving variable (arc in the cycle formed by the entering variable and the spanning tree), update the spanning tree, and update primal and dual variables until all dual slacks are non-negative.
- **Finding a Spanning Tree Basis:** A square submatrix of $\tilde{A}$ (incidence matrix without the root node row) is a basis if and only if the arcs corresponding to its columns form a spanning tree.
- **Calculating Primal Flows from a Spanning Tree:** Starting from leaf nodes, iteratively use flow balance equations at each node to determine flow values along arcs, working inward until all flows are determined.

- **Calculating Dual Variables from a Spanning Tree:** Starting from the root node, iteratively use the dual flow equation ($y_j - y_i = c_{ij}$) for arcs in the spanning tree to compute dual variables for each node.
- **Parametric Self-Dual Network Simplex Method:** Intertwine primal and dual pivots to reduce a perturbation parameter μ from ∞ to 0.
- **Linear Programming Formulation of Max Flow:** Maximize x_{ts} subject to flow conservation constraints at each node and capacity constraints on each arc, where x_{ts} represents the flow along a fictitious arc from the sink to the source with infinite capacity.
- **Duality:** Formulate a dual problem from a primal problem by transposing the constraint matrix and swapping the objective function coefficients with the right-hand side constants of the primal constraints. The optimal objective function values of the primal and dual problems are equal (Strong Duality).